

Direct Action Planning: An Auditable Action-Comparison Contract for Finite-Budget Resource Scheduling

ANONYMOUS AUTHOR(S)

Finite-budget resource schedulers must choose concrete capacity commands while preserving resources for uncertain future demand. When candidate effects, affordability, scores, and command delivery are recorded separately, verifying how an optimizer's choice becomes an executed action requires additional integration evidence. We introduce Direct Action Planning (DAP), an action-comparison contract that applies explicit resource effects to candidate states, evaluates affordable post-cost states under one causal forecast, and sends the feasible score maximizer directly to the actuator. We evaluate DAP on two public workload traces with matched mechanism controls and exact dynamic-programming diagnostics, then instantiate the same contract in a Kubernetes controller. Against a matched learned-transition control, explicit action effects raise completion by 0.65/3.79 percentage points while using 7.73/19.09 fewer resource units on Azure/GenTD26. Planning takes 0.81/0.95 ms per decision. The verification interface transfers to both DAP and PDS: it detects and localizes all 3,554 activated contract violations, with zero false positives in 640 clean contexts and a median checking cost of 7.80 microseconds. In matched Kubernetes controls, DAP raises Azure completion by 6.77 points and lowers Ready cost by 54.4% versus immediate scoring; 160 runs verify direct command mapping and sampled-ledger compliance. DAP makes planning decisions independently checkable across software boundaries while retaining efficient, action-aware resource allocation.

CCS Concepts: • **Computing methodologies** → **Planning and scheduling**; • **Software and its engineering** → *Software performance*.

Additional Key Words and Phrases: resource scheduling, finite budget, approximate dynamic programming, action-comparable planning, Kubernetes, autoscaling

ACM Reference Format:

Anonymous Author(s). 2027. Direct Action Planning: An Auditable Action-Comparison Contract for Finite-Budget Resource Scheduling. In *Proceedings of The ACM International Conference on the Foundations of Software Engineering (FSE 2027)*. ACM, New York, NY, USA, 21 pages.

1 Introduction

A production resource scheduler must do more than predict future demand. On every control cycle, it must choose one concrete capacity command, establish why that command is preferable to the other affordable candidates, and preserve enough resource for future demand. This action-level decision recurs in autoscaling, admission control, and queue management, where immediate action effects are often exposed by an API or calibrated model but future workload remains uncertain. A finite intervention budget couples the decisions over time: the central problem is action comparison under scarcity.

Existing approaches solve complementary parts of this problem through different runtime abstractions. Model-free policies and value functions learn long-run choices from interaction [37, 42, 46]; constrained reinforcement learning incorporates resource costs or policy constraints [1, 12, 40, 43, 52]; MPC exposes model structure through online lookahead [31]; and PDS/ADP separates controlled action effects from subsequent exogenous uncertainty [8, 25, 35]. What remains to be specified at integration time is a decision object that keeps candidate consequences, hard affordability, comparable scores, and the final actuator command connected and inspectable throughout one control cycle.

DAP treats action comparison itself as the deployed software interface. In each control cycle, it constructs explicit action-conditioned branches, supplies one shared causal forecast to every candidate, applies the external resource ledger before scoring, and sends the feasible maximizer

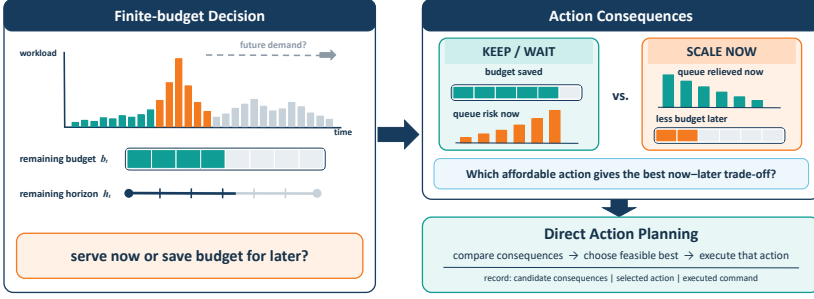


Fig. 1. Motivation for direct action planning. The schematic endpoints show how a lower-resource action preserves later opportunity, while a higher-resource action can improve current service at the cost of future budget. DAP compares the affordable consequences among four capacity levels and executes the best now-later trade-off.

directly to the actuator. The decision record retains each candidate's consequence, scoring charge, post-cost state, score, feasibility status, delivered command, and ledger update. Learning is confined to the exogenous forecast and downstream value, while action effects are supplied by explicit equations or frozen calibrations. This record connects three forms of evidence: service-resource frontiers evaluate productivity, exact dynamic programming tests action ordering, and live Kubernetes runs verify delivery and ledger fidelity. The same contract spans datasets and deployment environments even when fitted parameters differ. We position DAP as a runtime contract and verification interface built on the post-decision decomposition.

This paper makes three contributions:

- (1) **An auditable action-comparison contract.** DAP binds one causal forecast, explicit candidate effects, hard external affordability, the complete feasible score vector, and the delivered actuator command in a single runtime record. An executable fault study detects all 3,554 activated in-scope violations across DAP and PDS, with no false positives in 640 clean contexts and a median checking cost of 7.80 microseconds.
- (2) **Service-resource evidence for action ordering.** DAP combines budget-conditioned continuation with explicit action effects to compare affordable capacity choices under a shared causal context. It yields a Pareto-nondominated GenTD26 operating profile and a favorable Azure service-cost frontier. An exhaustive small-state diagnostic attributes a large part of the ordering error of a black-box transition to omitted action effects (69.33% lower Q-star regret and 99.49% fewer errors in the registered large-regret subset); this diagnostic isolates the mechanism on the registered grid, with workload-scale evidence reported separately.
- (3) **A deployable controller with verified execution.** DAP maps the same decision contract to the Deployment API, an independently measured Ready-time ledger, and direct command delivery. The deployed path achieves complete logged execution fidelity and zero violations of the sampled ledger under the registered lifecycle guard, while raising Azure completion by 6.77 percentage points and reducing Ready cost by 54.4% over immediate scoring under the same model and execution path.

2 Background and Motivation

2.1 Finite-Budget Resource Scheduling

At time t , a controller observes causal system state s_t , remaining budget b_t , and remaining horizon h_t . Candidate a denotes a resource level $C_t(a)$ for the next control interval. We distinguish three

cost objects: the branch charge $c_t^{\text{score}}(s_t, a)$ used to form the post-action value state, a conservative commitment $\bar{c}_t(s_t, a)$ used by the pre-actuation mask, and the measured ledger increment ΔC_t^{led} observed after actuation. The feasible set is

$$\mathcal{A}_t(b_t, s_t) = \{a \in \mathcal{A} \mid \bar{c}_t(s_t, a) \leq b_t\}. \quad (1)$$

After execution, the environment returns service reward r_t and applies

$$C_{t+1}^{\text{led}} = C_t^{\text{led}} + \Delta C_t^{\text{led}}, \quad b_{t+1} = B - C_{t+1}^{\text{led}}, \quad h_{t+1} = h_t - 1. \quad (2)$$

The objective is the finite-horizon discounted return

$$J = \mathbb{E} \left[\sum_{t=0}^{H-1} \gamma^t r_t \right], \quad (3)$$

subject to hard action affordability. In the trace environment, every candidate satisfies $c_t^{\text{score}}(s_t, a) = \bar{c}_t(s_t, a) = c(a)$. After execution, $\Delta C_t^{\text{led}} = c(a_t^{\text{exec}})$, so the update reduces to $b_{t+1} = b_t - c(a_t^{\text{exec}})$. In Kubernetes these three objects separate expected interval cost, a state-dependent safety reserve, and sampled Ready-Pod-time consumption. In both realizations, s_t, b_t, h_t form one 14-dimensional observation o_t .

We split the transition into an action-controlled component and an exogenous component. Let g contain the queue, capacity, service, cost, budget, and horizon equations, and let x_{t+1} denote future demand. Then

$$s_{t+1} = F(s_t, a_t, x_{t+1}) = f_{\text{exo}}(g(s_t, a_t), x_{t+1}). \quad (4)$$

The scheduling software already implements g : it knows the resource target, the action price, and represents how available capacity drains a queue through an equation or calibration. Learning a full black-box F discards that interface and asks data to identify effects that are already explicit. DAP instead learns only the uncertain quantity and the long-run value that cannot be computed in one step.

The trace constraint is pathwise rather than an expected-cost objective. With nonnegative exact token costs, $b_0 = B$, and $a_t \in \mathcal{A}_t(b_t, s_t)$, induction gives $\sum_t c(a_t) \leq B$. Value error can change which feasible action is chosen, but it cannot authorize an unaffordable trace action. The Kubernetes realization retains the hard pre-actuation mask while replacing exact tokens with a state-dependent commitment and a sampled Ready-time ledger; its conditional bound and empirical compliance evidence are specified in Section 4.

2.2 Why Finite Budgets Require Direct Action Comparison

Figure 1 summarizes the service–opportunity trade-off at the two ends of the affordable action set. Its *keep/wait* and *scale now* labels show the low- and high-resource endpoints, while DAP compares all four concrete capacity levels under the shared causal context. The runtime requirements below specify how this comparison becomes explicit and auditable.

2.3 Runtime Requirements and Execution Fidelity

These are requirements on the deployed decision path. They complement the optimization objective by making affordability, action comparison, and the actuator boundary observable in the same cycle.

Table 1. Causal observation contract used by the trace environments and the Kubernetes controller. Every transform is fitted on training data only.

Group	Fields and decision role
Demand	Current load, recent load, and burst intensity summarize the observed arrival process and drive the shared next-load forecast.
Queue/service	Queue, queue growth, active capacity, hot-resource ratio, utilization, mean/tail latency, SLO history, and resource pressure expose the known effect that an action can change.
Opportunity	Remaining-budget and remaining-horizon ratios condition the value of preserving resource for future decisions.

Definition 1 (Action-comparable execution contract). For one causal observation o_t containing normalized budget and horizon, let b_t also denote the corresponding raw, auditable external-ledger value. Given common descriptor $\hat{x}_{t+1}$, the decision record is

$$\begin{aligned}
 \mathcal{D}_t = \{ & o_t, b_t, \hat{x}_{t+1}, (\bar{c}_t(s_t, a))_{a \in \mathcal{A}}, \\
 & (\hat{o}'_{t,a}, c_t^{\text{score}}(s_t, a), r_t(a))_{a \in \mathcal{A}}, \\
 & (V_\theta(\hat{o}'_{t,a}), Q_{\text{DAP}}(o_t, a))_{a \in \mathcal{A}_t(b_t, s_t)}, (Q_{\text{DAP}}(o_t, a) = -\infty)_{a \notin \mathcal{A}_t(b_t, s_t)}, \\
 & a_t^{\text{plan}}, a_t^{\text{exec}}, \Delta C_t^{\text{led}}, C_{t+1}^{\text{led}}, b_{t+1} \}.
 \end{aligned} \tag{5}$$

A controller satisfies the contract when (C1) every candidate uses the same timestamped exogenous descriptor; (C2) its explicit action effect and exact or calibrated scoring charge construct the post-action state before continuation value is evaluated; (C3) an external ledger applies $\bar{c}_t(s_t, a)$ and removes unaffordable candidates before $a_t^{\text{plan}} = \arg \max_{a \in \mathcal{A}_t(b_t, s_t)} Q_{\text{DAP}}(o_t, a)$; and (C4) the delivered command, observed resource use, and next ledger state are recorded so that $a_t^{\text{exec}} = a_t^{\text{plan}}$ can be checked.

The implementation writes JSONL state, action, and event records keyed by control step. These contain raw and normalized observations with field timestamps and missing-data policies, one branch object per action, feasible bits and scores, the selected target, API latency, and sampled ledger intervals. A per-Pod metrics read is retried three times; if it still fails, that Pod is marked missing and omitted from service metrics while the independent Deployment Ready count continues to govern affordability. An uncaught planning or API error triggers base-target cleanup and marks the run incomplete. Replay checks shared-forecast equality, feasible argmax, target mapping, delivered identity, selected-action feasibility, and ledger closure. These are checkable runtime invariants for the registered controller lifecycle and fault suite.

That contract has four requirements, which map one-to-one to the modules in Figure 2: (R1) all candidates receive the same causal information about exogenous uncertainty; (R2) computable or calibrated action effects remain explicit rather than being relearned by a black-box model; (R3) remaining budget and horizon enter the continuation state after the branch charge; and (R4) the feasible ordering reaches the actuator without a second learned decision boundary.

These requirements separate three properties that are often conflated. *Feasibility* asks whether a decision respects the resource ledger. *Productivity* asks how much service the consumed resource produces. *Execution fidelity* asks whether the deployed action preserves the candidate ordering

Table 2. Native decision interfaces of common controller abstractions. “External” marks a field outside the method’s native decision object; DAP emits candidate consequences, affordability, scores, and actuation identity as one runtime record.

Abstraction	Candidate comparison	Budget semantics	Decision artifact	Actuator boundary
Learned policy controller	Policy logits or learned action values; explicit effects not intrinsic	External mask or learned constraint	Policy output	Integration layer
Lagrangian RL	Cost enters reward/dual update	Expected or penalized cost; hard mask external	Policy output plus multiplier	Integration layer
PDS/ADP	Exact controlled afterstate; exogenous effect in expectation	External constraint interface	Afterstate values	Selection and actuation may be external
MPC	Explicit online sequence tree under a model	Constraints inside optimizer	Plan can expose action or sequence scores	Deployment separately records first-command identity
Actor distillation	Planner supplies offline labels	Runtime mask external	Actor output can differ from teacher	Actor command
DAP	Shared forecast plus structured action branches	External pre-actuation feasible set	Complete feasible score vector	Logged feasible argmax delivered directly

intended by the value computation:

$$F_{\text{exec}} = \frac{1}{T} \sum_{t=1}^T \mathbf{1} \left[a_t^{\text{exec}} = \arg \max_{a \in \mathcal{A}_t(b_t, s_t)} Q_{\text{DAP}}(o_t, a) \right]. \quad (6)$$

Execution fidelity isolates planner-to-actuator correctness from value quality. Because the planner itself computes the argmax, F_{exec} is a delivery invariant; value quality is evaluated separately by productivity metrics and the exact-DP diagnostic. Overspend, observed API responses, effective targets, and F_{exec} audit whether the selected action remains feasible and reaches the actuator unchanged; completion, SLO, cost, and service–resource frontiers evaluate the productivity of that decision. This separation makes optimization quality and deployment integrity independently testable.

3 Direct Action Planning

Figure 2 realizes the contract as four modules: causal sensing, explicit candidate branches, affordable post-cost scoring, and direct actuation with ledger feedback. The trace evaluator and Kubernetes controller retain the same record through their respective actuators.

3.1 Module 1: causal sensing and common forecast

The controller forms the 14-dimensional observation in Table 1 from causal current/history system measurements together with the remaining budget and decision horizon. Current and recent load, burst intensity, queue pressure, capacity, service indicators, remaining budget, and remaining horizon are normalized with statistics fitted on the training split. The current reward uses observed load x_t ; only next exogenous load is unknown. A compact forecaster predicts it once,

$$\hat{x}_{t+1} = f_{\psi}(o_t), \quad (7)$$

and the same timestamped value is supplied to every candidate action. The action-invariant forecaster receives all 14 causal features and uses a $14 \rightarrow 32 \rightarrow 32 \rightarrow 1$ Tanh network with non-negative output. Training minimizes $\mathcal{L}_{\text{load}} + 4\mathcal{L}_Q + 2\mathcal{L}_{\text{rank}}$: Smooth-L1 log-load error, normalized feasible-branch Q error, and pairwise ranking loss with margin 0.02. Validation selects the epoch by action error plus 0.25 normalized Q error and 0.05 load error. The value is fitted first and frozen

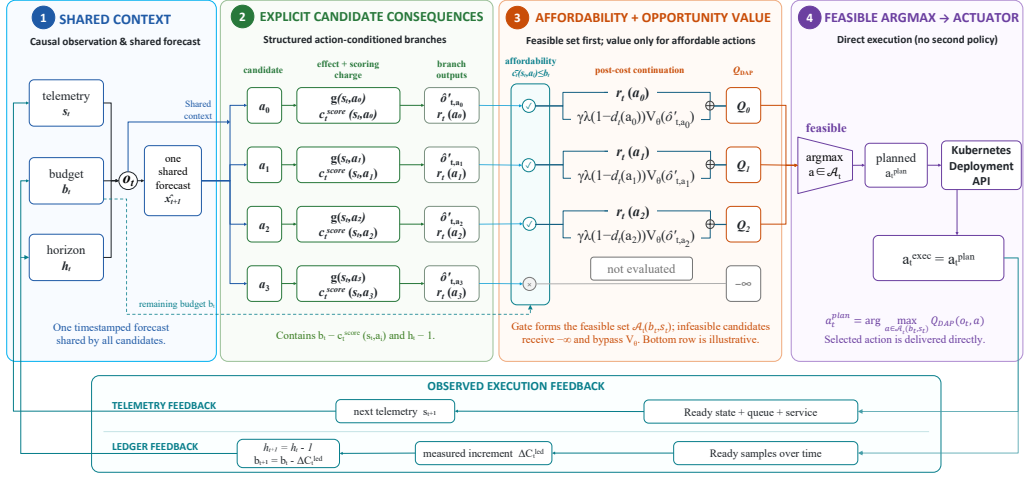


Fig. 2. The four-module DAP action-comparison contract. One observation and shared forecast feed explicit candidate consequences. External commitments form the feasible set before learned continuation-value evaluation; the maximizer reaches the Kubernetes Deployment API directly, and separate telemetry and ledger feedback close the loop.

during 24 forecast-training epochs. Targets $Q_i^{\text{target}}(a) = r_i(a) + \gamma(1 - d_i)V(o'_{i,a})$ use observed next-load branches; predictions substitute the common forecast. $\mathcal{L}_Q$ is Smooth-L1 error after dividing both Q vectors by the training feasible-Q standard deviation (floored at one). $\mathcal{L}_{\text{rank}}$ is the pairwise hinge loss on normalized Q differences, with the target ordering supplying the sign. Adam uses learning rate 10^{-3} , batches of 256, and gradient clipping at 5.

Sharing the forecast establishes a common basis for action comparison. Independent candidate forecasts would mix the endogenous effect of a resource action with exogenous prediction variation. One common forecast makes candidate differences attributable to the action, its cost, and continuation-value error. The decision record stores both the forecast and its observation timestamp.

3.2 Module 2: structured action-conditioned branches

The trace action is a one-step capacity level, not a persistent increment: $C_t(a) = C_{\text{base}} + \delta(a)$, with token costs (0, 1, 2, 4) and nominal extras (0, 1, 2.2, 4.8). The effect lasts for the current trace step and the next decision chooses a fresh level. Dataset-specific capacity calibration is fitted on training data and shared by all methods. In the Kubernetes instantiation the same ordered action identifiers map instead to absolute Deployment targets (1, 2, 3, 5), which persist until the next command; its legacy no_op identifier denotes the base/recovery target 1, so it requests scale-down when the current target is larger. For current arrival x_t , queue q_t , and candidate capacity $C_t(a)$, each copied branch applies

$$u_t = q_t + x_t, \quad y_t(a) = \min(u_t, C_t(a)), \quad (8)$$

$$q_{t+1}(a) = \text{clip}(u_t - y_t(a), 0, q_{\text{max}}), \quad (9)$$

followed by per-step clearance, a queue-derived latency proxy, and its SLO indicator:

$$\rho_t(a) = \frac{y_t(a)}{\max(u_t, 1)}, \quad (10)$$

$$\ell_t^{\text{tail}}(a) = 1 + \frac{2q_{t+1}(a)}{C_t(a)}, \quad (11)$$

$$z_t(a) = \mathbf{1}[\ell_t^{\text{tail}}(a) > \ell_{\text{SLO}}]. \quad (12)$$

The common simulator reward is

$$r_t(a) = \rho_t(a) - 0.04q_{t+1}(a) - 0.08\ell_t^{\text{tail}}(a) - 2z_t(a). \quad (13)$$

After inserting the shared forecast, the branch emits

$$\hat{o}'_{t,a} = \phi(s'_{t,a}(\hat{x}_{t+1}), b_t - c_t^{\text{score}}(s_t, a), h_t - 1). \quad (14)$$

Thus queue, service, cost, budget, and horizon differ only through the candidate action; explicit scheduler semantics are not relearned by a transition network.

3.3 Module 3: affordability and post-cost opportunity scoring

A $14 \rightarrow 64 \rightarrow 64 \rightarrow 1$ Tanh network learns the continuation value $V_\theta(o)$. Fitted value iteration regresses

$$y_i = \max_{a \in \mathcal{A}_i(b_i, s_i)} [r_i(a) + \gamma(1 - d_i)V_\theta(o'_{i,a})] \quad (15)$$

for 18 FVI target-update rounds indexed 0–17. At deployment, each affordable candidate receives

$$Q_{\text{DAP}}(o_t, a) = r_t(a) + \gamma\lambda(1 - d_t(a))V_\theta(\hat{o}'_{t,a}), \quad d_t(a) = \mathbf{1}[h_t = 1]; \quad (16)$$

unaffordable actions receive $-\infty$ and bypass the continuation-value call. The exact action cost therefore changes both feasibility and the budget-conditioned continuation state in the trace realization; Kubernetes uses the separate reserve defined below. Both deployed planners bypass V_θ on the final decision, and terminal-mask tests assert that no value call can reorder the $h_t = 1$ action scores.

Training has three non-overlapping chronological roles. The fitting block trains forecast and value candidates; the selection block chooses a retained value round and $\lambda \in \{0, 0.25, 0.5, 0.75, 1\}$; the development-evaluation block supports information-matched controls. The final temporal evaluator loads a frozen checkpoint and performs inference only. Selection retains the immediate planner ($\lambda = 0$) as a candidate and applies fixed validation guardrails: completion within 0.005, SLO violation within 0.01, and added cost no greater than 0.10*B* relative to that reference. Eligible candidates are ordered by return, completion, SLO, cost, and deterministic tie breaks. These rules select an operating point; they do not modify the deployed score.

For each fitting state, data collection copies the environment and executes every affordable action, storing its reward, next observation, feasibility, and terminal bit; 24 training and six validation episodes are collected per domain. Each FVI round freezes the preceding network as its target, computes the four masked Bellman branches, and fits the state maximum with Smooth-L1 loss using Adam (learning rate 10^{-3}), batches of 256, two epochs per round, and gradient norm 5. A DAP fitting state expands every affordable branch using the known transition equations; these branch evaluations are model computations, not additional environment interactions. The network width is 64, output scaling is fitted from training-reward RMS, and checkpoints $\{0, 2, 4, 6, 8, 10, 12, 14, 16, 17\}$ are the ten prespecified rounds retained for validation selection. Thus “18 rounds” denotes Bellman target updates, not 18 retained checkpoints or optimizer epochs. All

external learned baselines receive 102,400 environment interactions, the same 64-unit width, observation/action contract, temporal splits, and common hard safety mask. The native constraint losses remain active, but the mask prevents any method from spending an unaffordable action; this isolates productivity after safety admission and is a protocol choice. Complete search and training configurations are included in the frozen manifests.

3.4 Module 4: direct execution and accountable feedback

The controller executes the deterministically tie-broken feasible maximizer,

$$a_t^{\text{plan}} = \arg \max_{a \in \mathcal{A}_t(b_t, s_t)} Q_{\text{DAP}}(o_t, a), \quad (17)$$

and issues it directly, so $a_t^{\text{exec}} = a_t^{\text{plan}}$, without actor distillation or a second policy boundary. Every cycle records the causal observation, affordable set, common forecast, per-action immediate reward, continuation value, total score, delivered action, and updated ledger. The emitted decision and budget use can consequently be recomputed from the log.

For $|\mathcal{A}| = 4$, online work is one forecast plus $O(|\mathcal{A}|)$ one-step transition and value calls. A depth- K , width- W beam MPC performs $O(KW|\mathcal{A}|)$ branch expansions. DAP amortizes longer-horizon reasoning into V_θ while preserving explicit one-step action semantics and exposing an action-specific consequence trace. Section 4 maps this module to a Kubernetes Deployment actuator and an independently measured Ready-Pod-time ledger.

From post-decision value to a deployed action contract. PDS/ADP provides the core decomposition between an action-controlled afterstate and subsequent exogenous uncertainty [8, 25]. DAP advances this decomposition across the deployment boundary: it materializes every candidate under one shared causal forecast, applies external pre-actuation feasibility, evaluates the affordable post-cost branches, retains the complete score vector, and preserves the selected candidate's identity through actuation and ledger closure. This turns an effective value representation into an executable and testable action-comparison interface. Section 5.5 evaluates the nearest PDS/ADP representation under matched data and capacity, while Table 2 identifies the additional runtime artifacts exposed by DAP.

AI-assistance disclosure. A generative-AI coding assistant assisted with controller and adapter code, tests, experiment-record parsing, statistical analysis, exact-DP diagnostics, and figure/layout scripts. For the matched evaluation, it also proposed and implemented the experimental protocols, executed trace, GPU-training, and Kubernetes jobs on the research server, inspected their raw outputs, and drafted the resulting analysis and manuscript revisions. Protocols, code, checkpoints, failed attempts, and measurements are retained for verification. Numerical results are computed from the archived experimental measurements. The assistant also assisted with literature lookup and prose. The named authors remain responsible for the research design, verification of measurements and citations, interpretation, and final submitted content.

4 Kubernetes Realization

The simulator provides controlled action-level comparisons, but a resource controller must also map those actions into a real orchestration loop. We implemented DAP as an external Python controller for a local Kubernetes cluster. Each 5-second cycle reads Deployment status, Pod readiness, per-Pod Prometheus counters, queue depth, request rate, latency, CPU, SLO, and remaining budget. The planner maps its four actions to absolute replica targets $\{1, 2, 3, 5\}$ and issues a Deployment scale request through `kubectl`. A target remains in force until the next cycle; the legacy “no-op” identifier is explicitly the base/recovery action, not a hold-current command. Thus a base

action issued from target 5 requests scale-down. The controller does not modify the application or Kubernetes scheduler.

The system budget is denominated in Ready replicas above one base replica, not in requested replicas. A separate tracker samples actual Ready state approximately every 0.25 seconds and a pre-actuation reserve removes candidates that do not fit the remaining sampled ledger. After each run, the controller returns to the base target and continues accounting until scale-down is observed. Kubernetes readiness probes ensure that a new Pod is charged and used only once it becomes Ready; Pending or non-Ready Pods are excluded from Service traffic [45], while the target-based reserve still accounts for their possibility of becoming Ready during the next interval. This external control path uses the same Deployment scale interface exposed to Kubernetes' native periodic Horizontal Pod Autoscaler [44]; the difference is the finite-horizon Ready-time ledger and action-level planner placed before that interface. The design follows the controller-oriented lineage from Borg to Kubernetes [9, 48] without modifying either the scheduler or application.

Let $R(\tau)$ be the Ready-replica process and let one replica be the uncharged base capacity. The target continuous-time cost is

$$C_{\text{ready}}(t) = \int_0^t \max(R(\tau) - 1, 0) d\tau. \quad (18)$$

For observations (τ_k, R_k) , the implementation records the left-step approximation

$$\hat{C}_k = \sum_{j=1}^k \max(R_{j-1} - 1, 0)(\tau_j - \tau_{j-1}). \quad (19)$$

Every sample interval and Ready count is logged, so $\hat{C}_k$ is independently recomputable; requested replicas are logged separately. For control period Δ , current Ready count R_t , candidate absolute target $n(a)$, and calibrated lifecycle guard G , the mask uses

$$\bar{c}_t(R_t, a) = (n(a) - 1)_+ \Delta + \max\{(R_t - 1)_+, (n(a) - 1)_+\}G. \quad (20)$$

It admits a non-base target only when $\hat{C}_t + \bar{c}_t(R_t, a) \leq B$; the base target remains the recovery command. The reserve is conditional on a bounded control interval and a Ready tail covered by G , including outstanding replica commitments. Under these conditions, an admitted action plus return to base fits the remaining ledger. The implementation measures the left-step Ready integral, so the live budget endpoint is sampled-ledger compliance. Exact token affordability applies to the trace environment; continuous-time accounting additionally requires independently bounded command delays, Ready tails, and pending commitments.

The original controller used startup-derived guards $G = 3.2768$ seconds (Azure) and 3.2632 seconds (GenTD26). A direct lifecycle study measures 60 scale-up/return-to-base pairs across both profiles and targets 2, 3, and 5. The first five repetitions per profile–target calibrate the maximum observed Ready-withdrawal delay plus its sampling gap and a one-second margin; the next five validate that rule. The resulting guards are 3.068/3.015 seconds, and all 30 validation withdrawals fit them (maxima 1.847/1.867 seconds). Pod deletion is measured separately (maxima 2.530/2.549 seconds). These measurements use one shared node under background activity. The paired runs retain the larger original guard and continue accounting through observed return to base.

This architecture separates three concerns: live metric collection constructs the 14-field observation with provenance; the planner ranks feasible actions; and the actuator plus budget ledger enforce deployment and resource semantics. The runtime uses one timestamped snapshot per cycle. Missing Pod metrics are recorded and omitted from service features; the independent current Ready channel is read from Deployment status and remains separate from stale learned features

in the lag fault experiment. If the Deployment query itself fails, the run is marked incomplete and the cleanup path requests the base target. Consequently, service noise can affect ranking, while the Ready-Pod ledger and pre-actuation mask remain separate from the learned score.

The live implementation preserves the learned transition semantics: the value branch receives interval-average Ready capacity, while an independent current- Ready channel supplies the hard affordability mask. Utilization and EWMA features use the same causal windows as training. To keep burst estimates causal and action-comparable, the runtime computes one envelope for all actions:

$$\hat{x}_{t+1} = 1.10 \max(\hat{x}_{t+1}^{\text{neural}}, x_t, \hat{x}_{t+1}^{\text{EWMA}}).$$

The contract transfers across trace and Kubernetes instantiations with dataset- and deployment-specific fitted parameters. Trace checkpoints are fitted on each workload's training split and frozen before temporal testing. Kubernetes uses separate checkpoints and an independently measured Ready-capacity/startup/scale-down calibration, selects controller settings on chronological validation inputs, and freezes both before live test plans. The later GenTD26 continuation coefficient is likewise selected on disjoint validation plans; it does not change the state, actions, branch equations, hard mask, or actuator path.

The runtime is deliberately external to the scheduler. The workload driver owns a frozen request plan and client-side outcomes; the worker service owns admission, bounded queueing, CPU work, timeout accounting, and Prometheus metrics; Kubernetes owns Pod lifecycle and Service routing; and the controller owns state construction, planning, budget accounting, and replica patches. Only one controller is active in a run. A planner, checkpoint, or API failure triggers the base-replica cleanup path and prevents the run from being marked complete. This separation lets the experiment distinguish a planning error from an actuator or request-accounting error.

Observation records retain raw and normalized values, timestamps, freshness, and missing-data policies. Tests cover transition consistency, causal inputs, terminal values, absolute targets, stale safety data, and delivered argmax before formal runs are admitted to analysis.

5 Evaluation

We evaluate three questions:

RQ1 – Effectiveness: How does a finite-budget scheduler balance service quality and resource use across workloads and budget conditions?

RQ2 – Mechanism: Which decision mechanisms improve the timing and ordering of resource actions?

RQ3 – Deployment: Can planned actions be executed faithfully with measured budget compliance and practical control latency in a live system?

5.1 Datasets and chronological protocol

Azure Functions 2019 provides per-minute invocations in HTTP and asynchronous domains [32, 39]; days 1–8, 9–11, and 12–14 form train, validation, and test. GenTD26 is drawn from the Alibaba clusterdata trace and aggregated into ten-minute serving-demand bins [28]; its split is 60%/20%/20%. Each validation segment is further partitioned chronologically into 40% model validation, 30% checkpoint selection, and 30% development evaluation. Clipping, normalization, and capacity calibration use training data only. Each dataset fits its own controller, while the shared object is the four-module contract. Temporal slices are frozen inference re-evaluations of previously accessed project data; code, checkpoint, and window hashes bind their provenance.

The horizon is 64, $\gamma = 0.99$, and budgets are {48, 72, 96, 120, 144}. The main matrix contains two datasets, five budgets, and ten training seeds (820–829) with five evaluation episodes per domain.

All methods share observations, action semantics, rewards, trace windows, interaction budget, normalization, and the hard affordability mask. We report mask-binding frequency to quantify how often scarcity enters the decision path, alongside the zero-overspend invariant. GenTD26's domains are text-to-image and image-conditioned serving. Every method records zero token overspend under the common mask.

5.2 Comparators and statistics

External comparators include Double DQN, PPO, PPO-Lagrangian, CPO, P3O, Budgeted Fitted-Q, A2C, PID-Lagrangian, and a disclosed port of LCPO [1, 10, 23, 37, 40, 46, 52]. The baselines span model-free, constrained, budget-conditioned, and context-aware learning. Immediate planning, MPC-2/4/8, black-box transitions, actor distillation, budget/horizon removal, and PDS/ADP are information-matched controls. All methods share observations, interactions, actions, rewards, trace windows, normalization, and the affordability mask; native policy and dual losses remain unchanged. The mask is an explicit safety-admission protocol, so the comparison measures productivity after the same feasibility decision. DAP and MPC expose computable effects, while model-free methods infer them. Frozen manifests record optimizer settings, update counts, search ranges, branch counts, and wall-clock summaries.

The trained seed is the trace unit. We use 20,000 paired seed-block bootstrap resamples for 95% intervals, paired Cohen's d_z , wins/ties/losses, two-sided Wilcoxon tests, and Benjamini-Hochberg correction [2, 6, 7]; cost matching stays within observed budget frontiers. Kubernetes uses paired request plans, paired-bootstrap intervals, exact sign-flip tests, and the registered Holm family, retaining all seeds and failures. Matched trace, learned-model, and same-initialization controls use exact sign flips and Holm families of 24, 16, and eight endpoints; the matched live family contains 18 endpoints. Budgets are averaged within each of ten paired seed or plan blocks. The matched studies enumerate all 2^{10} sign assignments and use bootstrap seed 20261002; intervals are point-wise and tests carry familywise correction.

5.3 Metrics and artifact integrity

Return is the optimization objective. Episode completion is $\sum_t y_t / \max(\sum_t x_t, 1)$, whereas ρ_t is the per-step clearance term used only in reward. With initial queue zero, overflow $d_t = (q_t + x_t - y_t - q_{t+1})_+$ makes the conservation identity $\sum_t x_t = \sum_t y_t + \sum_t d_t + q_H$ explicit; unfinished and dropped work are not completions. Trace latency and SLO are queue-model proxies, not measured request percentiles. Kubernetes completion counts successful HTTP requests over all scheduled requests; latency percentiles condition on successful responses. The matched live study also reports the fraction of all requests that fail or exceed the SLO, counting fast failures as violations. Trace tables average episodes equally, assigning zero completion to zero-demand episodes (0/500 Azure and 20/500 GenTD26 episodes per method). A descriptive request-weighted check gives DAP completion 96.82%/79.56%; GenTD26 additionally has 5.15% dropped and 15.30% unfinished arriving mass. The 5,000-episode audit closes mass conservation to 1.31×10^{-12} and finds no token overspend. These aggregation conventions leave the primary comparisons unchanged. Resource use is total action cost in the trace environment and additional Ready-replica-seconds in Kubernetes. We report budget utilization, overspend, Pareto cell counts, matched-frontier effects, action disagreement, Q-star regret, execution fidelity F_{exec} , and decision latency within their corresponding evaluation families.

The temporal evaluator loads frozen checkpoints for inference. Manifests bind the dataset slice, configuration, checkpoint, code, and output hashes. The main trace matrix completed 100/100 cells;

Table 3. Mean application operating points over five budgets and ten trained seeds. Arrows show direction; bold marks the best value in each dataset and column, and † marks a Pareto-nondominated row over all four outcomes.

Dataset	Method	Return ↑	Completion ↑	SLO ↓	Cost ↓
Azure	DAP †	23.783	.9711	.1038	73.8
Azure	Double DQN†	14.295	.9736	.1186	62.9
Azure	PPO	6.715	.9574	.1967	91.3
Azure	A2C	-8.861	.9374	.2887	92.9
Azure	PPO-Lagrangian	6.465	.9561	.2068	92.5
Azure	CPO	20.021	.9684	.1200	86.0
Azure	P3O	.431	.9551	.2251	90.3
Azure	PID-Lagrangian	8.033	.9578	.1977	92.2
Azure	Budgeted Fitted-Q	-36.729	.9183	.4341	95.9
Azure	LCPO	7.833	.9523	.2044	93.8
GenTD26	DAP †	-105.195	.8851	.1479	39.0
GenTD26	Double DQN	-150.959	.8191	.2006	75.2
GenTD26	PPO	-215.266	.6979	.2691	95.6
GenTD26	A2C	-208.719	.7174	.2715	89.4
GenTD26	PPO-Lagrangian	-222.110	.6903	.2716	96.0
GenTD26	CPO	-162.225	.8122	.2101	79.4
GenTD26	P3O	-221.485	.6905	.2711	96.0
GenTD26	PID-Lagrangian	-219.204	.6930	.2706	96.0
GenTD26	Budgeted Fitted-Q	-199.016	.7322	.2604	93.0
GenTD26	LCPO	-170.505	.7913	.2268	83.0

PDS completed 100/100 development and 100/100 temporal cells; and the original Kubernetes studies completed 260/260 runs; the matched live extension adds 160/160 runs and 5,120 verified action records. Every registered seed and outcome appears in its corresponding table. This artifact contract makes each number in the paper traceable without treating repeated steps or requests as additional independent samples. For each primary paired comparison, the replication package retains the paired mean difference, 95% seed- or plan-block interval, Cohen's d_z , wins/ties/losses, and corrected test result; the compact main tables show the corresponding means and the figures show the intervals. Kubernetes reports plan-level rather than request-level uncertainty.

5.4 Service–resource effectiveness

Table 3 and Figure 3 show complementary service–resource regimes. On GenTD26, DAP has the best mean return, completion, SLO violation, and cost among the compared methods. Relative to Double DQN, it gains 45.764 return and 6.61 completion points while reducing SLO violation by 5.27 points and cost by 36.186 units. Relative to CPO, the corresponding improvements are 57.031 return, 7.29 completion points, 6.22 SLO points, and 40.346 cost units. Every trained-seed block favors DAP, and all four corrected comparisons have $q = 0.001953$.

Azure exposes two operator-relevant regimes. Against CPO, DAP improves return by 3.762 while reducing SLO violation by 1.62 points and cost by 12.144 units. At supported cost-matched points, DAP exceeds the interpolated CPO return frontier by 12.046 and reduces SLO violation by 5.98 points in 45/50 cells. DAP and the cost-lean Double DQN point form distinct Pareto choices: DAP provides higher return and lower SLO violation when additional budget is available. Against

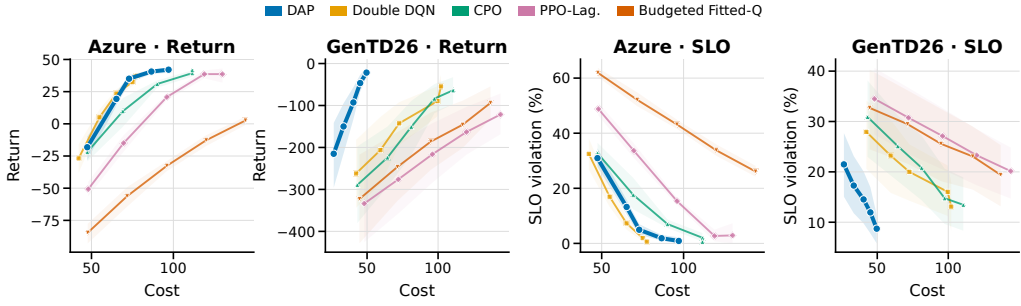


Fig. 3. Public-workload service-cost frontiers. The first two panels show return (higher is favorable) and the last two show SLO violation (lower is favorable), each against observed cost for Azure and GenTD26. Each point is one registered budget averaged over ten independently trained seeds; translucent ribbons are seed-bootstrap 95% intervals for the vertical endpoint, and lines connect observed budgets only.

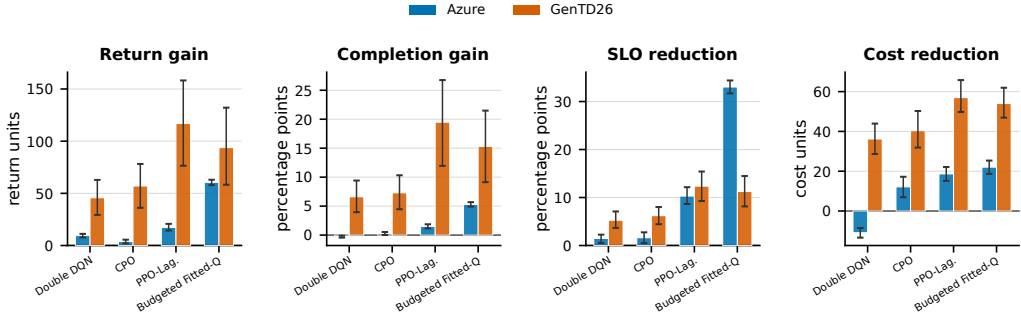


Fig. 4. **DAP improvement over baseline.** Define $\Delta R = R_{\text{DAP}} - R_{\text{baseline}}$ and $\Delta C = C_{\text{DAP}} - C_{\text{baseline}}$; SLO reduction is $\text{SLO}_{\text{baseline}} - \text{SLO}_{\text{DAP}}$, and cost reduction is $\text{Cost}_{\text{baseline}} - \text{Cost}_{\text{DAP}} = -\Delta C$. The four endpoints are therefore oriented so that positive bars favor DAP. Whiskers are paired-bootstrap 95% CIs over ten trained seed blocks after averaging five budgets; completion and SLO use percentage points, while return and cost retain native units.

the separately ported LCPO mechanism, all four paired outcomes favor DAP in every trained-seed block on both datasets ($q = 0.001953$).

Figure 4 reports the corresponding paired effects after averaging budgets within each trained seed.

Finding 1. DAP converts finite budgets into two operator-relevant regimes: a GenTD26 mean operating profile that improves all four outcomes relative to every comparator, and an Azure frontier that improves return, SLO, and cost over CPO while offering a return-SLO operating point alongside cost-lean model-free control.

5.5 Mechanisms of action ordering

RQ2 isolates action ordering through matched controls, exhaustive exact DP, and a PDS/ADP comparator.

Matched temporal controls cover all 100 dataset-budget-seed cells, with the original checkpoints and matched windows. Table 4 shows that Azure DAP improves return, completion, and

Table 4. Temporal mechanism controls over all five budgets and ten paired training seeds per dataset. Differences are DAP minus comparator; † instead compares the full-feature retrain with its identically initialized masked-feature counterpart. Return includes pointwise 95% intervals; completion and SLO differences use percentage points. Stars indicate Holm-adjusted $p < .05$ within each fixed study family (24 trace, 16 learned-model, eight matched-feature endpoints).

Dataset	Comparator	Δ return [95% CI]	Δ comp.	Δ SLO	Δ cost
Azure	Immediate	+8.72[+7.32, +10.20]*	+1.11*	-4.52*	-3.46
	PDS/ADP	+0.73[-0.24, +1.62]	+0.04	-0.24	-0.80
	MPC-8	+4.80[+3.98, +5.68]*	+0.69*	-2.39*	-3.46
	Learned trans.	+5.38[+3.81, +7.01]*	+0.65*	-2.54*	-7.73*
	No B/H†	+4.68[+1.72, +8.10]	+0.32	-2.07	-0.65
GenTD26	Immediate	+1.79[-0.13, +4.25]	+0.20	-0.04	-0.24
	PDS/ADP	+0.47[-0.58, +1.65]	+0.07	+0.04	-0.00
	MPC-8	-0.82[-2.51, +0.85]	+0.08	+0.22	-0.07
	Learned trans.	+19.30[+11.48, +27.98]*	+3.79*	-2.07*	-19.09*
	No B/H†	-1.91[-9.11, +7.65]	-0.65	+0.25	+7.26

Table 5. Exhaustive exact-DP action-ordering diagnostic over every feasible state-budget-horizon-action tuple. Lower Q-star regret is favorable.

Planner branch	Action agreement	Q-star regret
True transition + learned value	.9553	.0191
Black-box learned transition	.8805	.0613
Structured action effects	.9460	.0188

proxy SLO over immediate planning and MPC-8 after correction. Against a learned full transition with the same frozen continuation weight, both datasets improve on all four endpoints. DAP's mean decision times are 0.81/0.95 ms, versus 144.88/189.04 ms for MPC-8 in this run.

Removing budget/horizon features requires retraining. We therefore also train a full-feature control with the same initialization, data, batches, GPU, and selected round as the masked value. Table 4 reports this paired control alongside the frozen-checkpoint comparisons. The learned transition has width 96 and trains for 24 epochs; feature controls retain 18 FVI rounds, two epochs per round, and the frozen continuation weight. Training uses PyTorch 2.11 on an RTX 5090; planning runs on CPU. Frozen selection chooses zero continuation in 8/50 Azure and 16/50 GenTD26 checkpoints; those cells reduce to immediate scoring.

Exact DP separately evaluates every feasible state-budget-horizon-action tuple.

For state s , budget b , horizon h , and candidate a , backward DP defines $Q^*(s, b, h, a)$ and regret $Q^*(s, b, h, a^*) - Q^*(s, b, h, \hat{a})$, with the lowest-index exact maximizer a^* . True-transition, black-box, and structured branches reach .9553/.8805/.9460 agreement and .0191/.0613/.0188 regret; the structured branch reduces Q-star regret 69.33% and large-regret errors 99.49%. Table 5 reports the exhaustive ordering diagnostic.

The diagnostic fixes horizon 16, arrivals $\{1, 2, 4\}$, queues 0–6, budgets 0–12, costs $(0, 1, 2, 3)$, capacities $(1, 2, 3, 4)$, and $\gamma = .99$, with stable, burst, and periodic load scenarios. Backward DP evaluates the full feasible state-budget-horizon-action grid over three scenarios and five seeds; ties use the lowest action index.

The large-regret set is the upper quartile of positive Q-star regret among black-box-model disagreements: threshold 0.7073, 1,365 states, and 1,365 versus seven errors for black-box and structured branches. The 69.33% and 99.49% are exhaustive-grid ratios, while bootstrap uncertainty is reported for 1,800 rollouts. This grid quantifies the ordering mechanism; workload-scale behavior is reported in the trace and live evaluations, with independent paired Kubernetes runs.

Registered PDS/ADP nearest neighbor. We include a preregistered post-decision-state comparator. It applies the exact action, reward, queue, budget, and horizon transition, then removes next-load fields before the exogenous value is revealed. The afterstate network scores feasible actions as immediate reward plus discounted post-decision value. It shares the frozen DAP backbones, data, widths, candidate rounds, continuation weights, guardrails, budgets, and seeds; all development and temporal cells use the same code and contract hashes and complete with zero overspend.

The methods select different actions in 3.69–13.47% of Azure states and 1.09–4.11% of GenTD26 states. Their complete temporal contrasts appear in Table 4. The PDS adapter also provides a portability test: the same execution interface carries a different value representation while preserving its clean scores and actions. This separates the reusable verification capability from the choice of continuation model.

Finding 2. Explicit action effects improve temporal outcomes over the matched learned-transition control and reduce exact-DP Q-star regret. On Azure, learned continuation further improves service over immediate planning and MPC-8. DAP combines these action-ordering gains with sub-millisecond mean planning, and its execution contract accommodates both DAP and PDS values.

5.6 Kubernetes execution and control overhead

Trace inference averages 0.630/0.613 ms (Azure/GenTD26), versus 46.46/37.84 ms for MPC-4 and 181.24/142.23 ms for MPC-8.

The Kubernetes runs use Prometheus telemetry, a five-second period, 32-cycle horizon, and 128/256 Ready-replica-second ceilings. DAP aligns model capacity with an independent current-Ready safety channel under a frozen forecast envelope.

Four protocols cover scheduler comparison, fault runs, a binding GenTD26 replay, and a same-plan HPA/KEDA comparison. Configurations are frozen on disjoint inputs; the single local cluster and paired plans validate contract fidelity and characterize tested operating points. HPA/KEDA parameters and stabilization windows are fixed in the executable manifests and specified below.

The original studies contain 260 runs and 3,840 DAP decisions, all with exact request closure, restored base readiness, and delivered feasible argmax. No sampled-budget violation or controller deadline miss occurs. The mask activates in 33/120 runs and 248 decisions, removing 606 opportunities; mean inference/full-loop latency is 0.98/0.90 ms and 0.248/0.282 s, below 5 s. Deadlines are checked every run; tail latency and controller resource use are retained as artifact diagnostics for subsequent capacity analysis.

A deterministic verifier replays six boundaries (forecast, feasible argmax, delivery identity, target mapping, feasibility, and ledger closure). It finds no clean-record violation and localizes all six single-field mutations in the 3,200-record suite (19,200/19,200).

Executable faults and checker controls. An executable trace adapter injects faults before branch construction, selection, mapping, delivery, or ledger updates take effect. The seven fault types are stale cycle context, inconsistent forecast, incorrect post-cost budget, nonmaximal selection, target remapping, delivery override, and omitted ledger charge. The checker receives independently witnessed context and actuator receipts without fault labels. For example, a recorded decision selects capacity 6 with score 1.801; a delivery override executes the valid, affordable capacity 5 with score 1.785. Local validity checks accept either target, while the independent receipt exposes the

Table 6. Direct native-controller comparison on the tested Kubernetes cluster (means over ten paired plans). Cost is additional Ready-replica-seconds; SLO retains the original latency-only violation definition. Bold marks the favorable value in each profile and column.

Profile	Method	Completion $\uparrow$	SLO $\downarrow$	Ready cost $\downarrow$
Azure	DAP	.9654	.3705	48.37
Azure	HPA	.9490	.6157	84.33
Azure	KEDA	.9520	.4116	119.58
GenTD26	DAP	.9723	.0690	227.17
GenTD26	HPA	.8265	.2002	96.46
GenTD26	KEDA	.8727	.2440	59.81

change in planner-to-actuator identity. All 640 clean DAP/PDS decision contexts preserve the original scores and actions. On independently witnessed activated faults, the cross-boundary checker detects and localizes 1,940/1,940 DAP and 1,614/1,614 PDS violations; local validity, bounds, and affordability assertions detect 6/3,554, while passive logs perform no automatic detection. Clean false positives are 0/640; median checker-only time is 7.80 microseconds (95th percentile 8.69), excluding evidence collection and persistence. A separate +10% coherent-capacity control yields 50 changed actions in 640 contexts while preserving record consistency, and the checker correctly retains those internally consistent records. Each injected run executes one copied context; inactive cases and PDS forecast injections are excluded from the applicable-fault denominator. The executable specification is implemented through cross-boundary assertions, with the same checks supporting both planning representations.

In the original scheduler comparison, Azure DAP raises completion by 7.97 points (95% CI [6.48, 9.52]) and reduces Ready cost by 49.53 replica-seconds (CI [-64.75, -35.18]) over MPC-4. Under binding GenTD26, completion rises 3.68 points (CI [1.40, 6.15], Holm-adjusted $p = 0.0264$), with a mean cost difference of +0.049 (CI [-11.40, 10.39]). The two profiles provide service gains at lower or nearly equal mean cost; SLO intervals accompany both.

MPC-4 and Threshold were selected on a disjoint 180-run screen. The native-controller study evaluates DAP, HPA, and KEDA on common frozen plans (Table 6). Both use one to five replicas and zero scale-up/down stabilization. HPA targets 65% CPU with a two-Pod/10-second scale-up policy; KEDA polls queue depth every 10 seconds, with threshold four, a 20-second cooldown, and a two-Pod/5-second scale-up policy. Both allow 100% scale-down per five seconds. The native controllers reserve 45 seconds for recovery; DAP uses the calibrated 3.2768/3.2632-second guards. These rows describe the complete configured service-resource operating points.

Table 6 exposes all service-resource operating points. On Azure, DAP uses 35.96/71.21 fewer Ready-replica-seconds than HPA/KEDA, with completion .9654 and latency-SLO violation .3705. GenTD26 DAP provides a .9723-completion, .0690-SLO operating point at 227.17 Ready-replica-seconds. The four paired cost contrasts have Holm-adjusted $p = .0234$; the service means describe the configured operating points ($p = .0625$ –.2227).

To separate value, lookahead, and reserve effects, a matched protocol fixes the model and execution path and pairs DAP, immediate scoring, MPC-4, and a 45-second DAP reserve at budgets 128 and 256, using ten request plans per profile (160 completed runs). Plans map 32 consecutive trace bins to 32 five-second intervals using a training-fitted rate conversion and ten fixed activity strata. Plan seeds are 2026100201–2026100210; method order rotates across seeds and reverses at the second budget, whose order also alternates. Azure/GenTD26 use frozen continuation weights 1/.05 and HTTP SLOs .25/1 seconds. MPC uses depth four and zero terminal value; the 45-second

Table 7. Matched live controls: means over ten paired plans, averaging budgets 128 and 256 within each plan. Completion and failure-or-SLO count all scheduled requests. Cost is sampled Ready-replica-seconds; binding is the percentage of decisions with a masked candidate. Full budget-specific results and all 18 corrected contrasts appear in the replication package.

Profile	Method	Complete (%)	Fail/SLO (%)	Cost	Binding (%)
Azure	DAP	92.98	61.10	61.98	0.00
Azure	Immediate	86.22	53.27	135.93	19.69
Azure	MPC-4	88.56	47.96	133.13	14.53
Azure	DAP (45 s)	92.90	60.62	56.88	57.66
GenTD26	DAP	93.70	28.02	98.79	14.06
GenTD26	Immediate	89.05	37.24	82.34	10.62
GenTD26	MPC-4	90.11	36.51	82.07	10.78
GenTD26	DAP (45 s)	92.74	36.42	78.99	66.72

arm changes DAP's mask reserve alone. The shared kind node runs Kubernetes 1.35.1 and Python 3.11.6. Each worker requests .25 CPU/96 MiB and is limited to one CPU/256 MiB, with one-second readiness probes and a bounded queue/CPU-work HTTP service.

Matched live attribution. Table 7 holds the calibrated model and execution path fixed. Averaged over both budgets within each of ten paired Azure plans, DAP increases completion over immediate scoring by 6.77 points (95% CI [5.62, 7.88]) and reduces cost by 73.95 Ready-replica-seconds (CI [-85.51, -62.96]), a 54.4% saving. Against matched MPC-4, cost falls by 71.15 (CI [-85.86, -56.68]), or 53.4%. All three effects have Holm-adjusted $p = .0352$ across the fixed 18-endpoint family. An exact-run-ID scheduling amendment retained all completed runs; the one interrupted preparation generated no requests or actions. Omitting the two pre-amendment plan seeds gives an eight-plan sensitivity that retains the Azure completion-cost pattern (+6.79 points and -70.83 seconds versus immediate). The 45-second control exposes the reserve's effect on the feasible set: Azure/GenTD26 binding rises from 0/14.06% to 57.66/66.72%. Across all 160 runs, 3,390,344 requests reconcile exactly and all 5,120 logged actions maximize their feasible score vectors with the specified target mapping; every sampled ledger recomputes exactly, with zero budget violations and controller deadline misses.

DAP preserves its registered service bounds under deterministic 10% sample-and-hold dropout on both profiles, while the independent Ready channel remains intact. Across dropout, one-cycle observation lag, and five-second readiness delay, all 60 fault runs preserve deadline, delivery, cleanup, restoration, and sampled-ledger invariants.

Robustness to calibrated action effects. DAP remains stable when explicit action-effect estimates are perturbed. Across 96,000 chronological development decisions, $\pm 5\%$ changes at most 6.91% of actions and 0.56 completion points; $\pm 10\%$ changes 10.55% and 0.62 points. The mask records zero trace-token overspend. This diagnostic isolates capacity-effect error; forecast, lifecycle-guard, and cluster-shift sensitivity are separate evaluation dimensions.

Finding 3. DAP turns action-comparable planning into an auditable Kubernetes control path. The 160-run matched extension establishes an Azure completion-cost gain over immediate scoring and lower cost than matched MPC-4, alongside 5,120 verified feasible-argmax mappings and zero sampled-ledger violations. The original 260 runs additionally cover native controllers and telemetry/readiness faults. Together with the executable fault study, these results connect efficient planning, independently checkable execution, and measured resource allocation.

6 Discussion and Applicability

The PDS adapter reuses the candidate, commitment, command, and receipt fields with its original scores. Cross-boundary checks connect these fields: independently valid targets can still violate the selected action's identity. Model-quality diagnostics and execution checks therefore have complementary roles in the same evaluation workflow.

Deployment adapts two components: the action-effect calibration and the resource ledger. The tested realization uses one shared cluster, paired HTTP plans, sampled Ready accounting, and directly measured lifecycle reserves. Porting retains the causal comparison and receipt checks while calibrating capacity and recovery timing for the destination. Within the conditions in Section 4, the reserve couples each admitted action to its return to base. The trace realization supplies exact token accounting. Together these realizations make the interface useful for both discrete resource allocations and measured service capacity. Boundary-specific diagnostics connect delivery, accounting, and selection failures to the corresponding software component.

7 Related Work

Autonomic control and cloud resource management. Autonomic software frames adaptation as a monitored feedback loop [27]. Cloud controllers combine queue models, forecasting, and autoscaling: AutoControl, AutoScale, Autopilot, Sinan, and Autothrottle cover virtualized capacity and microservice SLOs [18, 34, 36, 50, 53]. Recent FSE systems add knowledge loops, predictive PID, content-aware signals, and LLM-assisted Kubernetes operations [11, 17, 49, 51]. They expose state and commands, whereas DAP records candidate consequences, budget commitment, comparable scores, delivered command, and actuator/ledger evidence. Kubernetes separates desired state, readiness, and reconciliation [9, 44, 45]; DAP makes the resulting boundaries auditable through forecast equality, feasible selection, target mapping, delivered identity, and ledger closure.

Cluster scheduling and learned policies. Cluster and serving systems optimize placement, packing, interference, and tail behavior [3, 4, 13–16, 19–22, 24, 26, 33, 38, 39, 47, 48]. DeepRM and Decima learn policies from interaction, while constrained methods add costs, trust regions, Lyapunov bounds, or penalty updates [1, 5, 12, 29, 30, 40, 43, 52]. Budgeted RL and LCPO address available budgets and changing contexts [10, 23]. These are strong policy and value baselines; DAP adds branch provenance, hard affordability, and planner-to-actuator identity.

Planning, post-decision value, and action ordering. Dyna, MPC, and ADP combine learning, constrained search, and value approximation [31, 35, 41]. Post-decision-state methods separate the action-controlled transition from subsequent exogenous uncertainty and can impose structural value properties [8, 25]. DAP carries this decomposition into deployment: every affordable candidate receives one causal realization, an explicit post-cost state and score, and a preserved identity through argmax, actuation, and ledger closure. PDS/ADP and MPC isolate value and search components, while exact DP measures ordering directly.

8 Conclusion

Direct Action Planning connects explicit action effects, post-cost values, hard affordability, and direct execution in one auditable interface. Explicit branches improve completion and resource use over matched learned transitions, with sub-millisecond mean planning. The low-overhead checker detects and localizes execution inconsistencies across DAP and PDS, supporting reuse across value representations. Matched Kubernetes runs demonstrate Azure completion and cost gains together with faithful command delivery and sampled-ledger compliance. DAP makes finite-budget resource planning executable, inspectable, and replayable across software boundaries.

9 Data Availability

The anonymous replication package is available at <https://anonymous.4open.science/r/direct-action-planning-reproducibility-3642/>. It contains source code, frozen checkpoints, trace preprocessing, configurations, controller and environment definitions, and the manuscript's tables and figures. The evidence/ directory provides per-episode/per-plan outcomes, all 66 matched statistical contrasts, selected continuation weights, executable-fault records, 60 lifecycle measurements, runtime versions, checksums, and a script that recomputes the paired analyses. The main text specifies the methods, metrics, experimental conditions, and principal results; the repository provides the corresponding machine-readable evidence and detailed protocols. Azure acquisition and GenTD26's pinned source, preprocessing recipe, and access instructions are documented under data/; redistribution follows the source datasets' terms. A public release will be maintained upon acceptance.

References

- [1] Joshua Achiam, David Held, Aviv Tamar, and Pieter Abbeel. 2017. Constrained Policy Optimization. In *Proceedings of the 34th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 70)*. 22–31. <https://proceedings.mlr.press/v70/achiam17a.html>
- [2] Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron Courville, and Marc G. Bellemare. 2021. Deep Reinforcement Learning at the Edge of the Statistical Precipice. In *Advances in Neural Information Processing Systems*, Vol. 34. <https://proceedings.neurips.cc/paper/2021/hash/f514cec81cb148559cf475e7426eed5e-Abstract.html>
- [3] Istemi Ekin Akkus, Ruichuan Chen, Ivica Rimac, Manuel Stein, Klaus Satzke, Andre Beck, Paarijaat Aditya, and Volker Hilt. 2018. SAND: Towards High-Performance Serverless Computing. In *2018 USENIX Annual Technical Conference*. USENIX Association, 923–935. <https://www.usenix.org/conference/atc18/presentation/akkus>
- [4] Omid Alipourfard, Hongqiang Harry Liu, Jianshu Chen, Shivaram Venkataraman, Minlan Yu, and Ming Zhang. 2017. CherryPick: Adaptively Unearthing the Best Cloud Configurations for Big Data Analytics. In *14th USENIX Symposium on Networked Systems Design and Implementation*. USENIX Association, 469–482. <https://www.usenix.org/conference/nsdi17/technical-sessions/presentation/alipourfard>
- [5] Eitan Altman. 1999. *Constrained Markov Decision Processes*. Chapman and Hall/CRC, Boca Raton, FL.
- [6] Andrea Arcuri and Lionel Briand. 2011. A Practical Guide for Using Statistical Tests to Assess Randomized Algorithms in Software Engineering. In *Proceedings of the 33rd International Conference on Software Engineering*. 1–10. doi:10.1145/1985793.1985795
- [7] Yoav Benjamini and Yosef Hochberg. 1995. Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing. *Journal of the Royal Statistical Society: Series B (Methodological)* 57, 1 (1995), 289–300. doi:10.1111/j.2517-6161.1995.tb02031.x
- [8] Mahesh Ganesh Bhat, Shana Moothedath, and Prasanna Chaporkar. 2026. Post-Decision State-Based Online Learning for Delay-Energy-Aware Flow Allocation in Wireless Systems. *arXiv preprint arXiv:2601.03108* (2026). <https://arxiv.org/abs/2601.03108>
- [9] Brendan Burns, Brian Grant, David Oppenheimer, Eric Brewer, and John Wilkes. 2016. Borg, Omega, and Kubernetes. *Commun. ACM* 59, 5 (2016), 50–57. doi:10.1145/2890784
- [10] Nicolas Carrara, Romain Laroche, Tanguy Urvoy, Olivier Pietquin, and Raphaël Fonteneau. 2019. Budgeted Reinforcement Learning in Continuous State Space. In *Advances in Neural Information Processing Systems*, Vol. 32. <https://arxiv.org/abs/1903.01004>
- [11] Brell Peclard Sanwouo Chekam, Clément Quinton, and Paul Temple. 2025. Breaking the Loop: AWARE is the New MAPE-K. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*. 626–630. doi:10.1145/3696630.3728512
- [12] Yinlam Chow, Ofir Nachum, Edgar Duenez-Guzman, and Mohammad Ghavamzadeh. 2018. A Lyapunov-Based Approach to Safe Reinforcement Learning. In *Advances in Neural Information Processing Systems*, Vol. 31. <https://proceedings.neurips.cc/paper/2018/hash/4fe5149039b52765bde64beb9f674940-Abstract.html>
- [13] Daniel Crankshaw, Xin Wang, Guilio Zhou, Michael J. Franklin, Joseph E. Gonzalez, and Ion Stoica. 2017. Clipper: A Low-Latency Online Prediction Serving System. In *14th USENIX Symposium on Networked Systems Design and Implementation*. USENIX Association, 613–627. <https://www.usenix.org/conference/nsdi17/technical-sessions/presentation/crankshaw>
- [14] Jeffrey Dean and Luiz André Barroso. 2013. The Tail at Scale. *Commun. ACM* 56, 2 (2013), 74–80. doi:10.1145/2408776.2408794

- [15] Christina Delimitrou and Christos Kozyrakis. 2013. Paragon: QoS-Aware Scheduling for Heterogeneous Datacenters. In *Proceedings of the 18th International Conference on Architectural Support for Programming Languages and Operating Systems*. 77–88. doi:10.1145/2451116.2451125
- [16] Christina Delimitrou and Christos Kozyrakis. 2014. Quasar: Resource-Efficient and QoS-Aware Cluster Management. In *Proceedings of the 19th International Conference on Architectural Support for Programming Languages and Operating Systems*. 127–144. doi:10.1145/2541940.2541941
- [17] Chiming Duan, Tong Jia, Minghua He, Pei Xiao, Lingzhe Zhang, Zhewei Zhong, Xin Zhang, and Ying Li. 2026. AIMS: A Content-Aware Resource Management Approach for AI Assistant Systems. In *Proceedings of the 34th ACM International Conference on the Foundations of Software Engineering*. 1232–1236. doi:10.1145/3803437.3805562
- [18] Anshul Gandhi, Mor Harchol-Balter, Ram Raghunathan, and Michael A. Kozuch. 2012. AutoScale: Dynamic, Robust Capacity Management for Multi-Tier Data Centers. *ACM Transactions on Computer Systems* 30, 4 (2012), 1–26. doi:10.1145/2382553.2382556
- [19] Ionel Gog, Malte Schwarzkopf, Adam Gleave, Robert N. M. Watson, and Steven Hand. 2016. Firmament: Fast, Centralized Cluster Scheduling at Scale. In *12th USENIX Symposium on Operating Systems Design and Implementation*. USENIX Association, 99–115. <https://www.usenix.org/conference/osdi16/technical-sessions/presentation/gog>
- [20] Robert Grandl, Ganesh Ananthanarayanan, Srikanth Kandula, Sriram Rao, and Aditya Akella. 2014. Multi-Resource Packing for Cluster Schedulers. In *Proceedings of the 2014 ACM Conference on SIGCOMM*. 455–466. doi:10.1145/2619239.2626334
- [21] Robert Grandl, Srikanth Kandula, Sriram Rao, Aditya Akella, and Janardhan Kulkarni. 2016. GRAPHENE: Packing and Dependency-Aware Scheduling for Data-Parallel Clusters. In *12th USENIX Symposium on Operating Systems Design and Implementation*. USENIX Association, 81–97. https://www.usenix.org/conference/osdi16/technical-sessions/presentation/grandl_graphene
- [22] Arpan Gujarati, Reza Karimi, Safya Alzayat, Wei Hao, Antoine Kaufmann, Ymir Vigfusson, and Jonathan Mace. 2020. Serving DNNs like Clockwork: Performance Predictability from the Bottom Up. In *14th USENIX Symposium on Operating Systems Design and Implementation*. USENIX Association, 443–462. <https://www.usenix.org/conference/osdi20/presentation/gujarati>
- [23] Pouya Hamadani, Arash Nasr-Esfahany, Malte Schwarzkopf, Siddhartha Sen, and Mohammad Alizadeh. 2025. On-line Reinforcement Learning in Non-Stationary Context-Driven Environments. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=l6QnSQizmN>
- [24] Benjamin Hindman, Andy Konwinski, Matei Zaharia, Ali Ghodsi, Anthony D. Joseph, Randy Katz, Scott Shenker, and Ion Stoica. 2011. Mesos: A Platform for Fine-Grained Resource Sharing in the Data Center. In *8th USENIX Symposium on Networked Systems Design and Implementation*. USENIX Association. <https://www.usenix.org/conference/nsdi11/mesos-platform-fine-grained-resource-sharing-data-center>
- [25] Daniel R. Jiang and Warren B. Powell. 2015. An Approximate Dynamic Programming Algorithm for Monotone Value Functions. *Operations Research* 63, 6 (2015), 1489–1511. doi:10.1287/opre.2015.1425
- [26] Kostis Kaffes, Neeraja J. Yadwadkar, and Christos Kozyrakis. 2019. Centralized Core-Granular Scheduling for Serverless Functions. In *Proceedings of the ACM Symposium on Cloud Computing*. 158–164. doi:10.1145/3357223.3362709
- [27] Jeffrey O. Kephart and David M. Chess. 2003. The Vision of Autonomic Computing. *Computer* 36, 1 (2003), 41–50. doi:10.1109/MC.2003.1160055
- [28] Yanying Lin, Shuaipeng Wu, Shutian Luo, Hong Xu, Haiying Shen, Chong Ma, Min Shen, Le Chen, Chengzhong Xu, Lin Qu, and Kejiang Ye. 2025. Understanding Diffusion Model Serving in Production: A Top-Down Analysis of Workload, Scheduling, and Resource Efficiency. In *ACM Symposium on Cloud Computing*. 1–15. Dataset documentation at Alibaba clusterdata revision 0d0f3f1. doi:10.1145/3772052.3772206
- [29] Hongzi Mao, Mohammad Alizadeh, Ishai Menache, and Srikanth Kandula. 2016. Resource Management with Deep Reinforcement Learning. In *Proceedings of the 15th ACM Workshop on Hot Topics in Networks*. 50–56. doi:10.1145/3005745.3005750
- [30] Hongzi Mao, Malte Schwarzkopf, Shaileshh Bojja Venkatakrishnan, Zili Meng, Zongheng Jerry Yang, and Mohammad Alizadeh. 2019. Learning Scheduling Algorithms for Data Processing Clusters. In *Proceedings of the ACM Special Interest Group on Data Communication*. 270–288. doi:10.1145/3341302.3342080
- [31] Ali Mesbah. 2016. Stochastic Model Predictive Control: An Overview and Perspectives for Future Research. *IEEE Control Systems* 36, 6 (2016), 30–44. doi:10.1109/MCS.2016.2602087
- [32] Microsoft Azure. 2020. Azure Functions Dataset 2019, Revision 2. Public workload trace. Accessed 2026-08-09. <https://github.com/Azure/AzurePublicDataset/blob/master/AzureFunctionsDataset2019.md>
- [33] Edward Oakes, Leon Yang, Dennis Zhou, Kevin Houck, Tyler Harter, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. 2018. SOCK: Rapid Task Provisioning with Serverless-Optimized Containers. In *2018 USENIX Annual Technical Conference*. USENIX Association, 57–70. <https://www.usenix.org/conference/atc18/presentation/oakes>

- [34] Pradeep Padala, Kai-Yuan Hou, Kang G. Shin, Xiaoyun Zhu, Mustafa Uysal, Zhikui Wang, Sharad Singhal, and Arif Merchant. 2009. Automated Control of Multiple Virtualized Resources. In *Proceedings of the 4th European Conference on Computer Systems*. 13–26. doi:10.1145/1519065.1519068
- [35] Warren B. Powell. 2011. *Approximate Dynamic Programming: Solving the Curses of Dimensionality* (2 ed.). John Wiley and Sons, Hoboken, NJ. doi:10.1002/9781118029176
- [36] Krzysztof Rzadca, Pawel Findeisen, Jacek Swiderski, Przemyslaw Zych, Przemyslaw Broniek, Jarek Kusmierek, Pawel Nowak, Beata Strack, Piotr Witusowski, Steven Hand, and John Wilkes. 2020. Autopilot: Workload Autoscaling at Google. In *Proceedings of the Fifteenth European Conference on Computer Systems*. 1–16. doi:10.1145/3342195.3387524
- [37] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal Policy Optimization Algorithms. *arXiv preprint arXiv:1707.06347* (2017). <https://arxiv.org/abs/1707.06347>
- [38] Malte Schwarzkopf, Andy Konwinski, Michael Abd-El-Malek, and John Wilkes. 2013. Omega: Flexible, Scalable Schedulers for Large Compute Clusters. In *Proceedings of the 8th ACM European Conference on Computer Systems*. 351–364. doi:10.1145/2465351.2465386
- [39] Mohammad Shahrad, Rodrigo Fonseca, Inigo Goiri, Gohar Chaudhry, Paul Batum, Jason Cooke, Eduardo Laureano, Colby Tresness, Mark Russinovich, and Ricardo Bianchini. 2020. Serverless in the Wild: Characterizing and Optimizing the Serverless Workload at a Large Cloud Provider. In *2020 USENIX Annual Technical Conference*. USENIX Association, 205–218. <https://www.usenix.org/conference/atc20/presentation/shahrad>
- [40] Adam Stooke, Joshua Achiam, and Pieter Abbeel. 2020. Responsive Safety in Reinforcement Learning by PID Lagrangian Methods. In *Proceedings of the 37th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 119)*. 9133–9143. <https://proceedings.mlr.press/v119/stooke20a.html>
- [41] Richard S. Sutton. 1991. Dyna, an Integrated Architecture for Learning, Planning, and Reacting. *ACM SIGART Bulletin* 2, 4 (1991), 160–163. doi:10.1145/122344.122377
- [42] Richard S. Sutton and Andrew G. Barto. 2018. *Reinforcement Learning: An Introduction* (2 ed.). MIT Press, Cambridge, MA.
- [43] Chen Tessler, Daniel J. Mankowitz, and Shie Mannor. 2019. Reward Constrained Policy Optimization. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=SkfrvsA9FX>
- [44] The Kubernetes Authors. 2026. Horizontal Pod Autoscaling. Accessed 2026-08-12. <https://kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale/>
- [45] The Kubernetes Authors. 2026. Liveness, Readiness, and Startup Probes. Accessed 2026-08-09. <https://kubernetes.io/docs/concepts/workloads/pods/probes/>
- [46] Hado van Hasselt, Arthur Guez, and David Silver. 2016. Deep Reinforcement Learning with Double Q-Learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 30. doi:10.1609/aaai.v30i1.10295
- [47] Vinod Kumar Vavilapalli, Arun C. Murthy, Chris Douglas, Sharad Agarwal, Mahadev Konar, Robert Evans, Thomas Graves, Jason Lowe, Hitesh Shah, Siddharth Seth, Bikas Saha, Carlo Curino, Owen O'Malley, Sanjay Radia, Benjamin Reed, and Eric Baldeschwieler. 2013. Apache Hadoop YARN: Yet Another Resource Negotiator. In *Proceedings of the 4th Annual Symposium on Cloud Computing*. 1–16. doi:10.1145/2523616.2523633
- [48] Abhishek Verma, Luis Pedrosa, Madhukar Korupolu, David Oppenheimer, Eric Tune, and John Wilkes. 2015. Large-Scale Cluster Management at Google with Borg. In *Proceedings of the Tenth European Conference on Computer Systems*. 1–17. doi:10.1145/2741948.2741964
- [49] Arthur Vitui and Tse-Hsun Peter Chen. 2026. LLM Agents for AIOps in Kubernetes: An Industrial Experience Report with Red Hat OpenShift. In *Proceedings of the 34th ACM International Conference on the Foundations of Software Engineering*. 678–689. doi:10.1145/3803437.3805240
- [50] Zibo Wang, Pinghe Li, Chieh-Jan Mike Liang, Feng Wu, and Francis Y. Yan. 2024. Autothrottle: A Practical Bi-Level Approach to Resource Management for SLO-Targeted Microservices. In *21st USENIX Symposium on Networked Systems Design and Implementation*. USENIX Association, 149–165. <https://www.usenix.org/conference/nsdi24/presentation/wang-zibo>
- [51] Pei Xiao, Lu Wang, Fangkai Yang, Guoqing Geng, Haoran Li, Jeff Zhu, Yu Kang, Yifan Li, Terry Chen, Yue Chen, Saravan Rajmohan, and Qi Zhang. 2025. Te-PID: An Adaptive Erasure Coding Temperature Management System for Optimized Cloud Storage. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*. 111–121. doi:10.1145/3696630.3728537
- [52] Linrui Zhang, Li Shen, Long Yang, Shixiang Chen, Xueqian Wang, Bo Yuan, and Dacheng Tao. 2022. Penalized Proximal Policy Optimization for Safe Reinforcement Learning. In *Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence*. 3744–3750. doi:10.24963/ijcai.2022/520
- [53] Yanqi Zhang, Weizhe Hua, Zhuangzhuang Zhou, G. Edward Suh, and Christina Delimitrou. 2021. Sinan: ML-Based and QoS-Aware Resource Management for Cloud Microservices. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. 167–181. doi:10.1145/3445814.3446693